

# FAITH



Workspace Platform · Progressive Web App

## FAITH PWA — Progress & Deployment Readiness

This document tracks two things: what has shipped so far (the changelog), and how ready the application is to run on real infrastructure (Dokploy, AWS EC2, or any other Docker-capable host).

---

VERSION

**v0.7.0**

PLATFORM

**Progressive Web App (PWA)**

# Development Progress

---

## v0.7.0

- Internal: eliminated all [any](#) types across the codebase — added a shared [IconName](#) type for icon names, a shared error-message helper for typed error handling, and typed route data instead of casting

## v0.6.0

- Login screen: reduced overall card height with a smaller logo, tighter spacing, and compact input fields
- Removed the fake clock from the desktop browser device-frame mockup's status bar
- Production web build now ships PWA install tags (manifest link, apple-touch-icon, viewport-fit=cover) that were previously silently dropped from the exported HTML
- Removed the internal-only "Latest Development" preview link from Settings -> About
- Updated Expo SDK dependencies to their expected versions

## v0.5.0

- Session restore failures on app launch now show an error message instead of failing silently
- Fixed a burst of unauthorized requests firing right after sign-out — attendance, leave, room, staff, and newflash no longer auto-refetch while there's no active session

## v0.4.0

- Home header avatar now shows a badge signaling it opens the account menu
- Removed the Push Notifications stub row from App Preferences
- Fixed the Dark Mode toggle not responding
- Removed leftover mock newflash data; Newsflash is now fully backed by the real API
- Room Booking image URLs now build from a shared constant instead of a hardcoded string
- Added a changelog to track version history

## v0.3.0

- Overlay Demo moved to an unlisted route, not linked from any in-app navigation
- Fixed toast, loader, modal, and sheet overlays rendering outside the desktop device frame
- About sheet "What's New" now shows only the latest version's changes instead of the full history

## v0.2.0

- App Layout: desktop browser now shows a phone device frame (bezel, notch, status bar, home indicator)

- Device frame reserves proper top/bottom safe-area space so app content never sits under the notch or home indicator
- Frame collapses when running as an installed PWA (standalone mode)

## v0.1.0

- Initial handover build
- Auth: sign-in/out with session expiry handling
- Attendance: daily status, weekly/monthly history
- Leave: apply, withdraw, balance, document & clinic support
- Newsflash: company broadcasts with acknowledgement
- Room Booking: availability, booking, and cancellation
- Staff: profile view and update

# Deployment Readiness

## Summary

FAITH PWA has moved off its earlier Vercel-based setup onto a fully containerized, cloud-native deployment: verified end-to-end, and migrated to the company's GitHub repository with complete commit history. The application builds and runs successfully as a Docker container. The only remaining step for a live deployment is provisioning the target infrastructure — a VPS for Dokploy, or an EC2 instance for AWS — which is a one-time infrastructure setup rather than further application work.

## Migration: From Vercel to Cloud-Native Deployment

The project's earlier deployment path was **Vercel**, using its `vercel.json` rewrites feature (`/api/:path*` -> the PHP backend) to proxy API calls from a statically-hosted frontend. That file still exists in the repository, but is no longer used — everything it did is now handled inside the Docker image itself.

This move to a self-hosted, containerized deployment was made for a few concrete reasons:

- **One artifact everywhere.** The same Docker image that runs locally (`docker compose up`) is the exact image that runs in production — no gap in behavior between a Vercel preview build and a live deployment.
- **No platform lock-in.** A generic container can run on Dokploy, a plain EC2 instance, or any other Docker host, instead of being tied to one vendor's build and rewrite pipeline.
- **API proxying moved from platform config into the app's own infrastructure.** nginx's `proxy_pass` inside the container now does what Vercel's `rewrites` used to do — documented directly in `nginx/default.conf.template`, so the proxy behavior travels with the app instead of living in a separate platform-specific config file.

- **Consistent with the target environment.** The organization's standard for this project is self-hosted, cloud-native deployment — Dokploy, with AWS EC2 as a viable alternative — rather than a serverless PaaS.

As part of this shift, the in-app "Latest Development" link that pointed to a Vercel preview build was removed from Settings -> About in v0.6.0. The `vercel.json` file itself remains in the repository as a leftover of the old setup and can be safely removed once the Dokploy/EC2 deployment is live.

## Containerization

The app is built as an **Expo Router web export**, served by **nginx** inside a single, lightweight container. The build happens in two stages: a `node:20-alpine` stage runs `expo export` to produce the static web bundle, and a second `nginx:1.27-alpine` stage serves only that bundle — Node and `node_modules` are discarded entirely, so the shipped image is small and has no JavaScript runtime dependencies at all.

nginx handles two responsibilities inside the container:

- **SPA routing** — any deep link (e.g. `/home/leave`) falls back to `index.html` so client-side routing works correctly on a hard refresh or direct link, instead of returning a 404.
- **API proxying** — requests to `/api/*` are proxied to the existing PHP backend, so the browser only ever talks to one origin and no CORS configuration is needed in production.

A default backend URL is baked into the image, so the container starts cleanly even if no environment variables are supplied at runtime — a resilience fix verified with a bare `docker run` and no environment configured.

## Repository

Source control has been fully migrated to the company GitHub organization with complete history preserved (148 commits), including the Dockerfile, Compose configuration, environment template, and deployment documentation.

| Item       | Value                                                                                                   |
|------------|---------------------------------------------------------------------------------------------------------|
| Repository | <code>github.com/maiman0/faithpwa</code>                                                                |
| Branch     | <code>main</code>                                                                                       |
| Contains   | Source code, full history, Dockerfile, docker-compose.yml, .dockerignore, .env.example, deployment docs |

## Environment Configuration

| Variable                         | When it applies | Purpose                                                                                           |
|----------------------------------|-----------------|---------------------------------------------------------------------------------------------------|
| <code>EXPO_PUBLIC_API_URL</code> | Build time      | Base API path, compiled into the JS bundle — cannot change without a rebuild                      |
| <code>BACKEND_URL</code>         | Run time        | Where nginx proxies <code>/api/*</code> — can change per-environment without rebuilding the image |

Build-time and run-time configuration are intentionally separated: the same built image can be pointed at a different backend simply by changing an environment variable at container start, without touching the frontend bundle.

## Status Checklist

| Task                                                 | Status                              |
|------------------------------------------------------|-------------------------------------|
| Migrate API proxy off Vercel to nginx (cloud-native) | Done                                |
| Containerize application (Dockerfile, .dockerignore) | Done                                |
| Local deployment setup (docker-compose.yml)          | Done                                |
| Environment configuration (.env.example)             | Done                                |
| Deployment documentation                             | Done                                |
| Verified running end-to-end in Docker                | Done                                |
| Repository migration (history + assets)              | Done                                |
| Remove leftover <code>vercel.json</code>             | Pending cleanup                     |
| Live cloud deployment                                | Pending infrastructure provisioning |

## Deployment Platforms

### Dokploy (Primary Target)

Dokploy is a self-hosted deployment platform that builds and runs the container directly from the GitHub repository — no manual image pushing required. It is the platform this project has been prepared and validated against.

Once a VPS with Docker and Dokploy is available, going live is a connect-and-deploy operation:

1. Create an Application in Dokploy and connect this repository ([main](#) branch), build type **Dockerfile**.
2. Set environment variables — [EXPO\\_PUBLIC\\_API\\_URL](#) (also as a build argument) and, optionally, [BACKEND\\_URL](#).
3. Forward the domain/proxy to container port [80](#) and enable HTTPS.
4. Deploy. Future updates redeploy automatically on push if auto-deploy is enabled.

```
GitHub repository -> Dokploy -> Docker build -> Container (nginx :80) ->
Live PWA (domain + SSL)
```

**Current blocker:** deploying through Dokploy requires a VPS/server to run it on, which is an infrastructure decision outside application development. No further code or configuration work is needed on the application side once a server is provisioned.

### AWS EC2 (Alternative Option)

Because the application is fully containerized, the same Docker image is portable to any Docker-capable host — including a plain EC2 instance, without any code changes:

1. Launch an EC2 instance (Amazon Linux or Ubuntu) with Docker installed, and an open security group rule for HTTP/HTTPS.
2. Build or pull the image on the instance and run it with the same environment variables used locally ([EXPO\\_PUBLIC\\_API\\_URL](#) at build time, [BACKEND\\_URL](#) at run time).
3. Put a reverse proxy or load balancer in front (e.g. an Application Load Balancer with an ACM certificate, or nginx/Caddy with Let's Encrypt) to terminate HTTPS and forward to container port [80](#).
4. Attach a domain via Route 53 (or any DNS provider).

A managed alternative on the same platform — ECS (Elastic Container Service) or Elastic Beanstalk — would remove the need to manage the EC2 instance directly, at the cost of a slightly more involved initial setup.

This path has not been executed yet, but requires no changes to the application itself — it's an infrastructure choice, not a development task.

### What's Needed to Go Live

Regardless of platform, the remaining steps are entirely infrastructure-side:

- A server (VPS for Dokploy, or an EC2 instance/ECS service for AWS) with Docker available.
- The two environment variables above, set for the target environment.
- A domain pointed at the server, with HTTPS enabled.
- A decision on auto-deploy vs. manual redeploy on future pushes.